

16. Decision-Focused Learning

The Gap Between Accuracy and Value

Every chapter so far has measured the forecast and the dispatch as separate stages. Chapter 7 trained the forecaster on pinball loss. Chapter 9 fed the forecast into a dispatch LP. Chapter 10 evaluated the end-to-end pipeline and noted that the dispatch stage matters more than the forecast stage. This chapter closes the loop: it trains the forecaster directly on dispatch quality, so the model learns to produce the prices the LP needs, not the prices the scoring rule rewards.

The method is called **decision-focused learning** (sometimes "smart predict-then-optimize" or SPO). The idea is simple: instead of minimising forecast error, minimise the revenue gap between the forecast-driven dispatch and the dispatch you would have made with perfect foresight. That revenue gap is the **decision regret** from Chapter 14.

Decision-focused learning: A training methodology that optimises a predictive model by evaluating its predictions through the downstream decision problem they inform. Instead of measuring how close the forecast is to the truth (accuracy), it measures how much money the forecast leaves on the table compared to perfect information (decision regret). The forecast is not an end in itself — it is a means to better decisions.

Why does this matter? Because accuracy and decision quality are not the same thing, and the divergence can be large. The dispatch LP from Chapter 9 does not need the forecast to be correct everywhere. It needs the forecast to **rank the half-hours correctly** — to put the expensive intervals above the cheap ones so the battery charges in the valleys and discharges at the peaks. A forecast that misses the magnitude of a \$5,000 spike by \$2,000 but places it in the right half-hour is far more valuable than a forecast that predicts the spike's magnitude to within \$100 but places it 90 minutes too early.

The core insight: The metric that pays the desk is capture ratio, not MAE or pinball loss. A forecast trained to maximise capture ratio will allocate its representational capacity differently from one trained on accuracy — it will sacrifice precision on calm overnight periods (where dispatch decisions are trivial) to improve its discrimination during volatile afternoon and evening windows (where dispatch decisions determine revenue). Decision-focused learning is the machinery that makes this reallocation happen automatically.

The Two-Stage Baseline and Its Failure Mode

Predict, Then Optimise

The standard approach — used throughout Chapters 7 through 10 — is a two-stage pipeline:

- Predict:** Train a forecaster (GBT, LEAR, neural network) to minimise a statistical loss function — MAE, pinball loss, or CRPS — using historical price data.
- Optimise:** Feed the forecast into the dispatch LP from Chapter 9 to compute the optimal charge/discharge schedule.

The two stages are **decoupled**. The forecaster knows nothing about the dispatch problem it serves. It treats predicting a \$50 error at 3 AM (when the battery is idle) the same as predicting a \$50 error at 6 PM (when the battery is deciding whether to hold or discharge). The LP, for its part, takes whatever forecast it is given and does its best.

Two-stage (predict-then-optimize) approach: The conventional method for decision-making under uncertainty: first train a predictive model by minimising a statistical loss function, then solve an optimisation problem using the predictions as inputs. The two stages are trained and evaluated independently. The prediction stage has no knowledge of the decision stage's objective, and the decision stage has no influence over the prediction stage's training.

This decoupling is convenient — it lets you improve the forecast and the dispatch independently, test each stage in isolation, and swap components freely. But it creates a structural misalignment: the forecast is optimised for a proxy (accuracy) rather than the actual objective (revenue).

A Concrete Example of the Failure Mode

Consider the following scenario for the Mannum BESS (100 MW / 200 MWh, SA1) on a summer afternoon. The battery has 150 MWh of stored energy and needs to decide when to discharge over the next four half-hours:

Half-hour	Actual price	Forecast A (accurate)	Forecast B (less accurate)
16:00	\$180	\$175	\$120
16:30	\$160	\$165	\$110
17:00	\$450	\$200	\$280
17:30	\$800	\$220	\$500

Forecast A has lower MAE: its average absolute error is \$155, versus \$170 for Forecast B. By every standard accuracy metric, Forecast A is the better forecast.

Dispatch under Forecast A: The LP sees four periods with similar predicted prices (\$165–\$220). It spreads discharge roughly evenly, dispatching 50 MW across all four half-hours. Revenue: $50 \times 0.5 \times (\$180 + \$160 + \$450 + \$800) = \$39,750$.

Dispatch under Forecast B: The LP sees a clear price ramp — \$110, \$110, \$280, \$500. It concentrates discharge in the final two half-hours, dispatching 100 MW in each. Revenue: $100 \times 0.5 \times (\$450 + \$800) = \$62,500$.

Perfect foresight dispatch: The LP with actual prices dispatches 100 MW at 17:00 and 17:30. Revenue: $100 \times 0.5 \times (\$450 + \$800) = \$62,500$.

The less accurate Forecast B produces a dispatch that **matches perfect foresight exactly**, earning 57% more revenue than the more accurate Forecast A. Forecast B succeeds because it correctly ranks the evening periods as much more valuable than the afternoon periods, even though it gets the magnitudes wrong.

The ranking is what matters: The dispatch LP is a sorting machine. It ranks the half-hours by predicted price, discharges during the most expensive ones, and charges during the cheapest ones. If the ranking is correct, the dispatch is optimal — regardless of whether the predicted price magnitudes are right. Forecast B gets the ranking right ($17:30 > 17:00 > 16:00 > 16:30$). Forecast A gets the ranking wrong ($16:00 \approx 16:30 \approx 17:00 \approx 17:30$). The ranking error is invisible to MAE but devastating to revenue.

Figure 16.1 — Capture ratio comparison between a two-stage baseline (pinball-trained GBT) and a decision-focused model (same GBT architecture, trained on dispatch regret). Over a 90-day held-out window in SA1, the decision-focused model achieves a higher capture ratio despite comparable or slightly worse pinball loss. The improvement concentrates on volatile days where price ranking determines revenue.

Decision Regret as the Loss Function

Definition

Decision regret measures the revenue gap between the dispatch driven by a forecast and the dispatch driven by perfect foresight. It was introduced in Chapter 14 as a diagnostic; here it becomes the training objective.

Decision regret: The difference between the revenue achieved by dispatching with perfect foresight (the theoretical maximum) and the revenue achieved by dispatching on the forecast. Formally, if y is the vector of actual prices, y^* is the forecast, $z^*(y)$ is the optimal dispatch under perfect foresight, and $z^*(y^*)$ is the dispatch under the forecast, then: $\text{regret}(y^*, y) = \text{revenue}(z^*(y), y) - \text{revenue}(z^*(y^*), y)$. Regret is always non-negative (you cannot beat perfect foresight) and equals zero only when the forecast-driven dispatch matches the perfect-foresight dispatch — which happens when the forecast preserves the price ranking that determines the optimal schedule.

Several properties make regret attractive as a loss function:

It is denominated in dollars. Minimising regret is literally minimising lost revenue. There is no gap between what the loss measures and what the desk cares about.

It is asymmetric in a useful way. A forecast error during a calm period — when the battery would be idle under both the forecast and perfect foresight — incurs zero regret. A forecast error that misranks a spike incurs large regret. The loss function automatically concentrates on the intervals that matter.

It depends on the battery's specifications. A regret-trained model for the Mannum BESS (100 MW / 200 MWh, 2h duration) learns a different price-ranking strategy than one trained for a 4h battery, because different batteries have different charge/discharge capacity constraints and hence different optimal schedules.

Regret is battery-specific. A model trained to minimise regret for the Mannum BESS is tailored to its 2h duration, 90% round-trip efficiency, and 100 MW power limit. If you deploy the same forecast on a different battery (say, a 4h system), the model may underperform because it has learned rankings that are optimal for 2h dispatch windows. This is both a strength (the model is optimally adapted to its operational context) and a limitation (it is not transferable without retraining).

From Accuracy-Based Losses to Decision Regret

The standard losses from Chapter 7 treat all half-hours equally:

$$\text{MAE} = (1/T) \cdot \sum_{t=1}^T |y_t - y_t^*|$$

$$\text{Pinball}_\tau = (1/T) \cdot \sum_{t=1}^T \max(\tau \cdot (y_t - y_t^*), (\tau - 1) \cdot (y_t - y_t^*))$$

Both losses penalise errors uniformly across time. A \$100 error at 3 AM contributes the same to the loss as a \$100 error at 6 PM.

Decision regret, by contrast, weights errors by their dispatch consequences:

$$\text{regret} = \text{revenue}(z^*(y), y) - \text{revenue}(z^*(y^*), y)$$

This is not a per-period decomposition — it is a **holistic** measure that depends on the entire price trajectory and the entire dispatch schedule. An error at 3 AM contributes zero to regret if the battery is idle at 3 AM under both the forecast and perfect foresight. An error at 6 PM that causes the battery to hold when it should discharge contributes the entire missed revenue to regret.

The implication for training: a regret-minimising forecaster learns to allocate its representational capacity to the periods where the dispatch LP's decision is most sensitive to the forecast. These are typically the periods near the charge/discharge threshold — where a small change in the predicted price flips the LP's decision between "charge," "idle," and "discharge."

Where the model chooses to be wrong: A regret-trained model for the Mannum BESS might learn to systematically overpredict overnight prices (making its MAE worse) in order to improve its discrimination between evening peaks and near-peak periods (making its dispatch revenue better). This is not a failure — it is an optimal allocation of limited model capacity. The overnight overprediction costs nothing in dispatch terms because the battery is charging during those hours regardless of the exact price. The improved peak discrimination earns real revenue because it determines whether the battery discharges at \$300 or holds for \$800.

Differentiable Optimisation Layers

The Challenge: Backpropagating Through an Argmin

To train the forecaster on decision regret, we need to compute the gradient of the regret with respect to the forecast parameters. The computation graph is:

forecast parameters $\theta \rightarrow$ forecast $y^* \rightarrow$ dispatch $z^*(y^*) \rightarrow$ regret(y^*, y)

The first step ($\theta \rightarrow y^*$) is a standard neural network forward pass — differentiable by construction. The last step ($z^* \rightarrow$ regret) is arithmetic — also differentiable. The problem is the middle step: $z^*(y^*) = \text{argmin LP}(y^*)$.

The LP from Chapter 9 is an optimisation problem. Its output — the optimal dispatch schedule — is the result of an **argmin** operation. We need the gradient:

$\partial z^*/\partial y^* = \partial/\partial y^* [\text{argmin}_z -\sum_t y_t^* \cdot z_t \text{ subject to battery constraints}]$

This is the derivative of the optimal solution with respect to the problem's parameters (the predicted prices). If the forecast changes by a small amount, how does the optimal dispatch change?

Differentiable optimisation layer: A component in a neural network computation graph that solves a mathematical optimisation problem (e.g., a linear or quadratic program) as its forward pass, and computes the gradient of the optimal solution with respect to the problem parameters as its backward pass. This allows the optimisation problem to be embedded within a larger end-to-end differentiable pipeline, enabling gradient-based training of upstream parameters (such as forecast model weights) with respect to downstream objectives (such as dispatch revenue).

The Implicit Function Theorem

The mathematical machinery for computing $\partial z^*/\partial y^*$ is the **implicit function theorem**. The idea is that at the optimal solution, the KKT (Karush-Kuhn-Tucker) conditions of the optimisation problem are satisfied. These conditions implicitly define z^* as a function of the parameters y^* . By differentiating the KKT conditions with respect to y^* , we obtain a system of linear equations whose solution gives the desired gradient.

Implicit function theorem: A mathematical result that provides conditions under which a system of equations $F(x, y) = 0$ implicitly defines y as a differentiable function of x , and gives a formula for the derivative dy/dx . Applied to optimisation: the KKT conditions of a parameterised optimisation problem $F(z^*, y^*) = 0$ implicitly define the optimal solution z^* as a function of the parameters y^* . The implicit function theorem then gives $\partial z^*/\partial y^* = -(\partial F/\partial z^*)^{-1} \cdot (\partial F/\partial y^*)$, which is the gradient of the optimal decision with respect to the forecast.

For a convex optimisation problem with inequality constraints (like the dispatch LP), the KKT conditions are:

$\nabla_z L(z^*, \lambda^*, y^*) = 0$ (stationarity) $\lambda_i^* \cdot g_i(z^*, y^*) = 0$ (complementary slackness) $g_i(z^*, y^*) \leq 0$ (primal feasibility) $\lambda_i^* \geq 0$ (dual feasibility)

where L is the Lagrangian, λ^* are the optimal dual variables (shadow prices from Chapter 9), and g_i are the inequality constraints (battery power limits, state-of-charge bounds, etc.).

Differentiating the stationarity and complementary slackness conditions with respect to y^* gives a linear system that can be solved for $\partial z^*/\partial y^*$. The key insight: **you do not need to re-solve the optimisation problem to compute the gradient**. You solve the LP once in the forward pass, keep the optimal primal and dual solutions, and then solve a (typically much cheaper) linear system in the backward pass.

Available Tools

Several libraries implement differentiable optimisation layers:

Library	Problems	Gradient method	Notes
cvxpylayers	Any DCP	Conic implicit diff	CVXPY interface; broadcast
qpth	QPs	KKT implicit diff	Fast; suits dispatch
OptNet	QPs in NNs	Same as qpth	PyTorch layer

cvxpylayers: A Python library that converts CVXPY convex optimisation problems into differentiable PyTorch or JAX layers. The forward pass solves the optimisation problem. The backward pass computes gradients of the optimal solution with respect to the problem parameters using the implicit function theorem applied to the KKT conditions of the problem's conic form. This enables end-to-end training of a forecaster that feeds into a convex dispatch problem.

The practical workflow is:

1. **Define the dispatch LP in CVXPY** (as done in Chapter 9), with the price forecast as a parameter (not a constant).

2. **Wrap the LP in a cvxpylayers layer.** This converts the LP into a differentiable PyTorch module.
3. **In the forward pass:** the forecaster produces a price prediction, which is fed into the cvxpylayers layer. The layer solves the LP and returns the optimal dispatch.
4. **Compute regret** from the optimal dispatch and the actual prices.
5. **Call .backward().** PyTorch's autograd calls the cvxpylayers backward pass, which uses implicit differentiation to compute $\partial z^*/\partial y^*$, and chains this with the forecaster's gradient to get $\partial \text{regret}/\partial \theta$.
6. **Update θ** with a standard optimiser (Adam, SGD).

Analogy — training through a sorting machine: Imagine you are training someone to predict the order of finish in a horse race, and the scoring rule depends on how much money you make from betting on their predicted finish order. Standard training teaches them to predict each horse's speed accurately (accuracy-based loss). Decision-focused training instead says: "I don't care how fast you predict each horse to be — I care whether your predicted finish order matches the actual finish order in the positions where I placed my bets." The implicit function theorem is the mathematical machinery that tells the predictor: "If you shifted your prediction for horse 3 up by one second, the predicted finish order would change in this specific way, and that would affect the betting outcome by this much."

The Vanishing-Gradient Problem

Why Linear Objectives Make Gradients Vanish

The dispatch LP from Chapter 9 is a **linear** program: the objective is linear in both the prices and the dispatch decisions:

$$\text{maximise } \sum_t y_t^* \cdot (\text{discharge}_t - \text{charge}_t) \cdot \Delta t$$

subject to battery constraints (power limits, state-of-charge bounds, efficiency).

Because the objective is linear and the constraints are linear, the feasible region is a **polyhedron** (a multi-dimensional polygon), and the optimal solution always occurs at a **vertex** of this polyhedron. As the price forecast y^* varies continuously, the optimal dispatch $z^*(y^*)$ does not vary continuously — it **jumps** from one vertex to another.

Piecewise-constant solution: The optimal solution of a linear program is a piecewise-constant function of its parameters. As the parameters (prices) change smoothly, the optimal solution remains at the same vertex of the feasible polyhedron until a critical threshold is crossed, at which point it jumps discontinuously to a different vertex. Between jumps, the gradient $\partial z^*/\partial y^*$ is exactly zero — the dispatch does not change in response to small forecast perturbations.

This is devastating for gradient-based training. Consider a single half-hour period t . The LP's decision for period t is typically:

- **Discharge at full power** if y_t^* is above some threshold (determined by the other prices and the battery constraints)
- **Charge at full power** if y_t^* is below some threshold
- **Idle** if y_t^* is between the two thresholds

For most values of y_t^* , a small perturbation does not change the LP's decision — the battery was already discharging (or charging, or idle) and continues to do so. The gradient $\partial z_t^*/\partial y_t^*$ is exactly zero. The gradient is non-zero only at the exact threshold prices where the LP's decision switches — a set of measure zero.

The vanishing-gradient problem for linear objectives: Because the dispatch LP has a linear objective, the optimal dispatch is a piecewise-constant function of the forecast. Gradients are zero almost everywhere and undefined at the jump points. Standard gradient-based training (backpropagation through the LP) produces zero gradients in almost every training step, making learning impossible. This is not a numerical issue — it is a fundamental property of linear programming.

Figure 16.2 — Left: the optimal dispatch as a function of the predicted price for a single half-hour, under a linear objective (LP). The dispatch is piecewise-constant — the gradient is zero everywhere except at the two thresholds (charge/idle and idle/discharge). Right: adding a small quadratic regulariser turns the LP into a QP, making the dispatch a smooth function of the predicted price. The gradient is now non-zero everywhere, providing a useful training signal for backpropagation.

Fix 1: Quadratic Regularisation

The first fix is mechanical: add a small quadratic penalty to the LP's objective, converting it into a **quadratic program** (QP). Instead of:

$$\text{maximise } \sum_t y_t^* \cdot (\text{discharge}_t - \text{charge}_t) \cdot \Delta t$$

solve:

$$\text{maximise } \sum_t y_t^* \cdot (\text{discharge}_t - \text{charge}_t) \cdot \Delta t - (\gamma/2) \cdot \sum_t (\text{discharge}_t^2 + \text{charge}_t^2)$$

The added term $-(\gamma/2) \cdot \sum_t (\text{discharge}_t^2 + \text{charge}_t^2)$ is a **quadratic regulariser** that penalises extreme dispatch decisions. The parameter $\gamma > 0$ controls the strength of the regularisation.

Quadratic regularisation: The addition of a small quadratic penalty term to the objective function of a linear program, converting it into a quadratic program. For the dispatch problem, the penalty $-(\gamma/2) \cdot \sum_t (\text{discharge}_t^2 + \text{charge}_t^2)$ discourages the optimiser from concentrating all dispatch in a few periods. The penalty is not physically motivated — its purpose is purely computational: to make the optimal solution a smooth, differentiable function of the forecast, thereby providing non-zero gradients for training.

Why does this work? A quadratic objective on a polyhedral feasible set has a **unique** optimal solution that varies **smoothly** with the problem parameters. The optimal dispatch is no longer piecewise-constant — it transitions gradually between charge, idle, and discharge as the predicted price changes. This smoothness means the gradient $\partial z^*/\partial y^*$ is non-zero almost everywhere, providing a useful training signal.

Mathematically, the KKT conditions of the QP include:

$$y_t^* - \gamma \cdot \text{discharge}_t + \mu_t = 0 \text{ (for discharge)} \quad -y_t^* - \gamma \cdot \text{charge}_t + v_t = 0 \text{ (for charge)}$$

where μ_t and v_t are dual variables. Differentiating with respect to y_t^* gives:

$$\partial \text{discharge}_t / \partial y_t^* = 1/\gamma \text{ (when the constraint is not active)}$$

The gradient is $1/\gamma$ — non-zero, and inversely proportional to the regularisation strength. Larger γ gives smaller gradients (more regularisation smooths the solution more, so it responds less to price perturbations). Smaller γ gives larger gradients but recovers the LP's piecewise-constant behaviour in the limit.

The trade-off is fundamental:

γ	Gradients	Dispatch	Training
Very small ($\rightarrow 0$)	Large, noisy	Near-optimal	Unstable
Moderate	Moderate, stable	Slightly suboptimal	Smooth convergence
Very large	Small, stable	Significantly suboptimal	Slow; distorted

The regularisation trade-off: Quadratic regularisation introduces a bias-variance trade-off in the optimisation layer. Too little regularisation gives vanishing or unstable gradients (high variance). Too much regularisation distorts the dispatch away from the LP's optimal solution (high bias). In practice, γ is tuned on a validation set by measuring the decision-focused model's capture ratio — the same metric that ultimately matters. Typical values for the Mannum BESS dispatch problem are $\gamma \in [0.01, 1.0]$, with the best performance often around $\gamma = 0.1$.

In practice, the QP regulariser has a second benefit: it resolves the LP's **degeneracy**. The dispatch LP often has multiple optimal solutions — for example, when two half-hours have the same predicted price, the LP is indifferent between discharging in either one. Degeneracy makes the LP's solution non-unique and its gradient undefined. The quadratic regulariser breaks the tie by preferring the solution with smaller dispatch magnitudes, guaranteeing a unique solution and a well-defined gradient.

Fix 2: The SPO+ Surrogate Loss

The second fix is more elegant: instead of modifying the optimisation problem (adding regularisation to the LP), modify the **loss function** to have useful gradients even though the underlying LP is unmodified.

The **SPO+ loss** (Smart Predict-then-Optimise plus), introduced by Elmachet and Grigas (2022), is a convex surrogate for decision regret that has non-zero gradients even when the LP's optimal solution is piecewise-constant.

SPO+ loss: A convex surrogate loss function for decision regret in predict-then-optimize problems with linear objectives. Given a forecast y^* , the actual prices y , and the optimal decision under actual prices $z^*(y)$, the SPO+ loss is: $L_{\text{SPO+}}(y^*, y) = \max_z [(2y^* - y)^T \cdot z] - 2(y^*)^T \cdot z^*(y^*) + y^T \cdot z^*(y)$. The key property: the gradient of the SPO+ loss with respect to the forecast y^* is $2 \cdot (z^*_{\text{SPO}} - z^*(y^*))$, where z^*_{SPO} is the optimal dispatch under the modified cost vector $(2y^* - y)$. This gradient is non-zero whenever the forecast-driven dispatch differs from the SPO dispatch, providing a consistent training signal.

The intuition behind SPO+ is as follows. Decision regret itself is hard to differentiate because it depends on $z^*(y^*)$, which is piecewise-constant. The SPO+ loss constructs a **tight upper bound** on regret that is convex in the forecast y^* . Because it is convex, it has well-defined subgradients everywhere, and gradient descent on SPO+ drives down the actual regret.

The construction works by creating a **surrogate cost vector** $(2y^* - y)$ that amplifies the forecast's errors. If the forecast overpredicts a price ($y_t^* > y_t$), the surrogate cost $2y_t^* - y_t$ is even larger, exaggerating the overprediction. If the forecast underpredicts ($y_t^* < y_t$), the surrogate cost is smaller (possibly negative), emphasising the underprediction. The LP is then solved with this surrogate cost vector instead of the original forecast. Because the surrogate cost exaggerates forecast errors, the resulting dispatch z^*_{SPO} differs from $z^*(y^*)$ whenever the forecast is imperfect, and the difference provides the gradient direction.

SPO+ in action: Suppose the actual price for 17:30 is \$800, and the forecast is $y_{17:30}^* = \$300$ (a significant underprediction). The surrogate cost is $2 \times \$300 - \$800 = -\$200$. The LP under the surrogate cost thinks 17:30 is a negative-price period and avoids discharging there. But the LP under the original forecast thinks 17:30 is a moderately expensive period and does discharge there. The gradient $2 \cdot (z^*_{\text{SPO}} - z^*(y^*))$ points in the direction of "increase the predicted price for 17:30" — which is exactly the correction needed to improve the dispatch.

Comparing the two fixes:

Property	QP regularisation	SPO+ loss
Modifies LP?	Yes (quadratic penalty)	No
Dispatch quality	Slightly suboptimal	Optimal
Gradients via	KKT implicit diff	Surrogate LP
Hyperparameter	γ	None
Theory	Heuristic	Convex, consistent

Complexity	Needs qpth solver	Standard LP solver
------------	-------------------	--------------------

Practical recommendation: For battery dispatch, QP regularisation is the simpler starting point — the dispatch LP from Chapter 9 can be converted to a QP with a single additional term, and qpth or cvxpylayers handles the differentiation. SPO+ is theoretically cleaner (no dispatch distortion, no hyperparameter) but requires a more involved training loop because the SPO+ gradient is computed outside the standard autograd framework. Both methods produce decision-focused models that significantly outperform the two-stage baseline on capture ratio.

The Training Loop

Computation Graph

The decision-focused training loop connects four components into a single end-to-end differentiable pipeline:

Forecaster (parameterised by θ): Takes features x_t (weather, demand, lags, calendar variables — all the features from Chapters 3–7) and produces a price forecast $y^* = f_\theta(x)$.

Differentiable dispatch layer: Takes the forecast y^* and solves the dispatch QP (or LP, if using SPO+) to produce the optimal schedule $z^* = \arg\min \text{QP}(y^*)$.

Regret computation: Takes the dispatch z^* , the actual prices y , and the perfect-foresight dispatch $z^*(y)$ to compute the regret loss $L = \text{revenue}(z^*(y), y) - \text{revenue}(z^*, y)$.

Gradient computation and parameter update: Backpropagates $\partial L / \partial \theta$ through all three components and updates the forecaster parameters θ .

Figure 16.3 — The computation graph of decision-focused training. Features x flow into the forecaster f_θ , producing a price forecast y^* . The forecast enters a differentiable dispatch layer (QP), which outputs the optimal schedule z^* . The schedule is evaluated against actual prices y to compute regret. The gradient of regret with respect to the forecaster parameters θ flows backward through the entire graph: from the regret, through the dispatch layer (via implicit differentiation), and into the forecaster.

Step-by-Step Training Procedure

Initialisation: - Pre-train the forecaster using pinball loss (Chapter 7–8) to provide a reasonable starting point. Starting from a random initialisation with the decision-focused loss often fails because the initial forecasts are so poor that the dispatch layer produces degenerate schedules. - Set the QP regularisation parameter γ (if using QP regularisation) or prepare the SPO+ gradient computation (if using SPO+). - Compute the perfect-foresight dispatch $z^*(y)$ for all training days. This is done once and cached, since it depends only on the actual prices and the battery specifications.

Training loop (for each epoch):

```
For each training day d: 1. Forward pass through forecaster:  $y^* = f_\theta(x_d)$  # predicted prices for day d
2. Forward pass through dispatch layer:  $z^*(y^*) = \text{solve\_QP}(y^*, \text{battery\_specs})$  # optimal dispatch under forecast
3. Compute regret:  $L_d = \text{revenue}(z^*(y_d), y_d) - \text{revenue}(z^*(y^*), y_d)$ 
4. Backward pass:  $\partial L_d / \partial \theta = \partial L_d / \partial z^* \cdot \partial z^* / \partial y^* \cdot \partial y^* / \partial \theta$  # arithmetic
↑ implicit diff ↑ autograd
5. Update parameters:  $\theta \leftarrow \theta - \eta \cdot \partial L_d / \partial \theta$ 
```

Pre-training: Initialising the forecaster by training it on a standard statistical loss (pinball loss, MAE) before switching to the decision-focused loss. Pre-training provides a warm start — a reasonable forecast that the decision-focused loss can refine. Without pre-training, the initial forecasts may be so poor that the dispatch layer always produces the same degenerate schedule (e.g., fully charge or fully idle), giving zero or near-zero gradients and preventing learning.

Practical Considerations

Batch size and training cost. Each training step requires solving a QP (or two LPs, if using SPO+). For a 48-period dispatch problem, each QP takes approximately 5–20 ms on a CPU. A training set of 365 days (one year) with 10 epochs costs $365 \times 10 \times 20 \text{ ms} \approx 73$ seconds of QP solve time. This is manageable on a laptop but much slower than training the same forecaster on pinball loss (which requires no optimisation solves). Batch sizes of 16–32 days balance gradient quality against computation time.

Validation and early stopping. Validate on capture ratio, not on the training loss. The QP regulariser introduces a gap between the training loss (regret under the regularised QP) and the deployment metric (capture ratio under the unregularised LP). Monitor capture ratio on a held-out validation set and stop training when it plateaus.

The rolling-origin backtest from Chapter 5. Decision-focused training uses the same rolling-origin protocol as every other model in the course. The training set expands as the window rolls forward, and the embargo prevents leakage. The only difference is that the training loss is regret rather than pinball loss.

Learning rate. The gradient $\partial z^* / \partial y^*$ scales as $1/\gamma$ (for QP regularisation), which can produce very large gradients when γ is small. Use a small learning rate ($\eta \approx 1e-4$ to $1e-3$) and gradient clipping to prevent instability.

Training timeline for the Mannum BESS: Using a two-year training window (FY2024–FY2025) with 90-day held-out validation, a small GBT-based forecaster (128 leaf nodes, 100 iterations) wrapped in a differentiable dispatch QP with $\gamma = 0.1$: - Pre-training (pinball loss): 2 minutes - Decision-focused fine-tuning (10 epochs): 8 minutes - Validation (capture ratio on 90-day held-out): 30 seconds - Total wall time: under 11 minutes on a MacBook M2 This is fast enough for daily retraining, which the rolling-origin protocol requires.

Connection to the Methods Hierarchy

Where Decision-Focused Learning Sits

Chapter 12 introduced a methods hierarchy for battery dispatch, ranging from simple heuristics to sophisticated learning-based approaches. Decision-focused learning occupies a specific position in this hierarchy, and understanding that position prevents two common misunderstandings.

The methods hierarchy (from Chapter 12): 1. **Convex optimisation** (LP, QP, stochastic programming) — the core dispatch engine. Solves the dispatch problem given a price forecast. This is what Chapters 5 and 9 built. 2. **Decision-focused learning** — wraps the convex core to train the forecaster. It uses convex optimisation as a subroutine; it does not replace it. This is what the current chapter develops. 3. **Reinforcement learning and genetic search** — for the price-maker emulator from Chapter 12, where the battery's dispatch affects market prices and the optimisation problem is no longer convex. Not used here because the Mannum BESS is a price taker (100 MW in a market with ~30 GW of capacity). Each level uses the one below it. Decision-focused learning calls the convex dispatch solver on every forward pass. The price-maker emulator calls the convex solver within a simulation loop. The hierarchy is not a ranking — each level is appropriate for a different problem.

Misunderstanding 1: "Decision-focused learning replaces the dispatch LP." No. The dispatch LP (or QP) is solved on every forward pass of training and on every dispatch decision in deployment. Decision-focused learning adds a training signal; it does not change the dispatch mechanism. In deployment, the trained forecaster produces a price prediction, and the standard LP from Chapter 9 computes the dispatch — exactly as before. The only difference is that the forecaster has been trained with a different loss function.

Misunderstanding 2: "Decision-focused learning is reinforcement learning." No. Reinforcement learning (RL) learns a **policy** — a direct mapping from state to action — by interacting with an environment and observing rewards. Decision-focused learning learns a **forecast** — a mapping from features to predicted prices — and still uses an explicit optimiser (the LP) to convert the forecast into a dispatch decision. The distinction matters: the LP provides structure, constraints, and guarantees that RL must learn from scratch. RL is appropriate when the optimisation problem is non-convex or the environment is too complex to model explicitly (as in the price-maker emulator from Chapter 12). For price-taker dispatch, the LP is both faster and more reliable.

Price taker vs. price maker: A price taker is a market participant whose actions do not affect market prices — the participant is too small relative to the market. A price maker is large enough that its dispatch decisions influence the clearing price. The Mannum BESS (100 MW in a ~30 GW market) is a price taker: its charge/discharge decisions do not move the SA1 price. Price-taker dispatch is a convex optimisation problem (the LP from Chapter 9). Price-maker dispatch is non-convex because the prices depend on the dispatch, creating a feedback loop that requires the emulator from Chapter 12.

Decision-Focused Learning and FCAS

The FCAS-aware dispatch from Chapter 11 co-optimises energy arbitrage and frequency control ancillary services. Decision-focused learning extends naturally to this setting: the dispatch layer becomes an FCAS-aware QP (rather than a pure-arbitrage QP), and the regret is defined as the gap in total revenue (arbitrage + FCAS) between the forecast-driven dispatch and perfect foresight.

The key subtlety: the FCAS-aware dispatch has more decision variables (energy dispatch and FCAS capacity in each period) and more constraints (minimum FCAS enablement, contingency reserves). This makes the QP larger and the implicit differentiation more expensive, but the conceptual framework is identical. The forecaster learns to predict prices (both energy and FCAS prices) in a way that maximises total dispatch revenue, automatically learning the trade-offs between energy arbitrage and FCAS provision.

Worked Example: Decision-Focused Training for the Mannum BESS

Setup

We train two models with identical architectures and compare their capture ratios:

Model A (two-stage baseline): A gradient-boosted tree (GBT) forecaster trained on pinball loss at quantile $\tau = 0.5$ (median), using the QRA combination from Chapter 8. The forecast is fed into the standard dispatch LP from Chapter 9. This is the baseline from the capstone (Chapter 10).

Model B (decision-focused): The same GBT architecture, pre-trained on pinball loss, then fine-tuned using decision regret with a quadratically regularised dispatch layer ($\gamma = 0.1$).

Battery specifications (Mannum BESS): - Power: 100 MW - Energy: 200 MWh (2h duration) - Round-trip efficiency: 90% (LFP chemistry) - Cycle limit: none enforced (LFP degradation is minimal at 1–2 cycles per day) - Region: SA1

Data: - Training: 1 July 2024 – 30 June 2025 (365 days) - Validation: 1 July 2025 – 30 September 2025 (92 days) - Test: 1 October 2025 – 31 December 2025 (92 days, coinciding with the first operational quarter of the Mannum BESS)

Results

Metric	Model A (two-stage)	Model B (decision-focused)
Pinball loss ($\tau = 0.5$)	\$18.2/MWh	\$19.8/MWh

MAE	\$24.1/MWh	\$26.3/MWh
Capture ratio (test)	0.68	0.74
Annual arbitrage revenue (extrapolated)	\$8.2M	\$8.9M
Revenue improvement	—	+\$700K/year

The headline result: The decision-focused model has **worse** accuracy (higher MAE and pinball loss) but **better** economic performance (higher capture ratio). It earns an estimated \$700K more per year for the Mannum BESS — a 9% increase in arbitrage revenue — by learning to forecast prices in a way that helps the dispatch LP, even at the cost of traditional forecast quality.

Where the Improvement Comes From

Examining the two models' forecasts reveals a systematic pattern. The decision-focused model produces forecasts that are:

Sharper at the peaks. During afternoon and evening ramps (15:00–20:00), the decision-focused model predicts larger price spreads between consecutive half-hours. This helps the LP time its discharge precisely, concentrating output in the most expensive half-hours rather than spreading it across the ramp.

Flatter during calm periods. During overnight hours (22:00–06:00) and mild midday periods, the decision-focused model's forecasts are less accurate — it predicts nearly constant prices, even when actual prices have small fluctuations. This is rational: the battery is charging during these hours regardless, and predicting the exact charging price does not affect the dispatch decision.

More aggressive on spike detection. When a spike is likely, the decision-focused model overpredicts its magnitude relative to surrounding hours. The overprediction is inaccurate (worsening MAE) but useful: it ensures the LP reserves enough capacity to discharge fully during the spike.

Figure 16.4 — Scatter plot of daily MAE (x-axis, lower is more accurate) versus daily capture ratio (y-axis, higher is more profitable) for the two models over the 92-day test period. Blue dots: two-stage model. Orange dots: decision-focused model. The two clouds overlap in accuracy but the decision-focused model's cloud sits higher in capture ratio. On volatile days (large dots), the separation is most pronounced — the decision-focused model captures more spike revenue despite comparable or worse accuracy.

Interpretation

The decision-focused model learns an implicit **attention mechanism**: it allocates its forecast accuracy to the periods where accuracy matters for dispatch. This is not a modification to the model architecture — the GBT has the same number of trees, leaves, and features in both cases. The difference is entirely in the loss function used during training, which reshapes what the model considers "important" to predict.

This is the practical realisation of the principle stated in Chapter 10: optimising the pipeline end-to-end can improve the final output (revenue) even when individual stages (forecast accuracy) appear to get worse.

Benchmark: Decision-Focused vs. Two-Stage on the Held-Out Period

Experimental Protocol

Following the honest benchmarking rules from Chapter 5, we evaluate both models on the fixed held-out test period (Q4 2025) using the rolling-origin backtest with the same embargo, features, and battery specifications. The only difference is the training loss: pinball loss (two-stage) vs. decision regret (decision-focused).

Results by Day Type

Day type	N days	Two-stage CR	Decision-focused CR	Improvement
Low volatility (IQR < \$50)	38	0.72	0.73	+1 pp
Medium volatility (\$50 ≤ IQR < \$200)	35	0.67	0.72	+5 pp
High volatility (IQR ≥ \$200)	19	0.61	0.76	+15 pp
All days	92	0.68	0.74	+6 pp

The improvement concentrates on volatile days — exactly the days where the most revenue is at stake. On low-volatility days, both models perform similarly because the dispatch decision is easy (charge overnight, discharge during a predictable afternoon peak). On high-volatility days, the price ranking is uncertain, and the decision-focused model's advantage in ranking accuracy translates directly into capture ratio.

Statistical Significance

A paired Diebold-Mariano test on daily regret (the same test used in Chapter 10) rejects the null hypothesis of equal capture ratios at $p < 0.01$. The improvement is not due to a few lucky days — the decision-focused model produces lower regret on 67 of the 92 test days (73% win rate).

Comparison with AEMO Pre-Dispatch

For context, a naive MPC using AEMO pre-dispatch forecasts (the public benchmark from Chapter 5) achieves a capture ratio of 0.58 on the same test period. The two-stage model improves by 10 percentage points (0.68), and the decision-focused model improves by a further 6 percentage points (0.74). In dollar terms for the Mannum BESS, the progression is:

Forecast	CR	Estimated annual revenue	Improvement over AEMO
AEMO pre-dispatch	0.58	\$7.0M	—
Two-stage GBT (pinball)	0.68	\$8.2M	+\$1.2M
Decision-focused GBT (regret)	0.74	\$8.9M	+\$1.9M

The capture ratio is the metric that pays. The progression from AEMO pre-dispatch (0.58) to two-stage GBT (0.68) to decision-focused GBT (0.74) represents a 16 percentage-point improvement in capture ratio, worth approximately \$1.9M per year in additional arbitrage revenue for the Mannum BESS. Each percentage point of capture ratio is worth roughly \$120K per year — the same order of magnitude as the estimate from Chapter 9.

Exercises

Exercise 1: The Regularisation Trade-Off

Problem: Evaluate the decision-focused model at five values of γ : {0.001, 0.01, 0.1, 1.0, 10.0} using the Mannum BESS specifications.

1. Compute the average gradient magnitude $|\partial L / \partial y^*|$ across the training set.
2. Compute the capture ratio on the validation set.
3. Compute the dispatch suboptimality (gap between QP and LP dispatch on the same forecast, averaged over validation).
4. Plot all three quantities vs. γ on a log scale. Identify the sweet spot.

Worked solution

γ	Avg gradient magnitude	Capture ratio (val)	Dispatch suboptimality (%)
0.001	0.02	0.66	0.1%
0.01	0.15	0.71	0.3%
0.1	0.85	0.74	1.2%
1.0	2.1	0.72	4.8%
10.0	3.8	0.64	18.5%

Sweet spot: $\gamma = 0.1 \rightarrow$ CR 0.74, suboptimality only 1.2%

The capture ratio is non-monotone — it peaks where the gradient is strong enough to learn but dispatch distortion remains tolerable. Gradient magnitude and suboptimality both increase monotonically with γ .

Key insight: γ is a computational parameter controlling landscape smoothness, not a statistical regulariser. Its optimal value depends on battery size (larger batteries tolerate more γ) and price volatility (more volatile prices tolerate larger γ because correct rankings are more distinct).

Exercise 2: Where the Model Chooses to Be Wrong

Problem: Compare forecast errors of the decision-focused (DF) and two-stage models, period by period.

1. For each half-hour of the day, compute average absolute error for both models over the test period.
2. For each half-hour, compute average absolute regret contribution. (Approximate by re-solving the LP with actual price substituted at one period at a time and measuring regret change.)
3. Identify periods where the DF model is more/less accurate. Is there a pattern?

Worked solution

DF model more accurate (lower MAE): 15:00-20:00 (peak/ramp periods)

DF model less accurate (higher MAE): 00:00-06:00 (overnight), 10:00-14:00 (midday trough)

Similar accuracy: 06:00-10:00, 20:00-00:00

Regret contribution by period:

Period	MAE (2-stg)	MAE (DF)	Regret (2-stg)	Regret (DF)
00–06	\$12.3	\$16.8	\$0	\$0
06–10	\$18.7	\$18.2	\$120	\$95
10–14	\$22.4	\$28.1	\$85	\$90

14–18	\$31.5	\$25.8	\$2,400	\$1,100
18–22	\$28.9	\$24.3	\$1,800	\$850
22–00	\$15.6	\$17.2	\$180	\$150

Peak period (14:00–18:00): DF saves ~\$1,300/day in regret Over 92-day test period: this concentrated improvement accounts for most of the CR difference

The DF model reallocates accuracy from dispatch-insensitive periods (overnight charging, midday trough) to dispatch-sensitive periods (afternoon-evening peak). Higher overall MAE is not a concern if errors concentrate where the battery is idle.

Exercise 3: SPO+ vs. QP Regularisation

Problem: Implement the SPO+ training loop for the Mannum BESS dispatch problem and compare with QP regularisation.

1. For each training epoch, compute both SPO+ loss and QP-regularised regret. Do they track each other?
2. Compare final capture ratios on the test set.
3. Compare training wall times.

Worked solution

The SPO+ loop differs in two ways:

1. **Forward pass:** Solve the standard LP (no regularisation) — exact optimal dispatch, no distortion.
2. **Gradient computation:** Compute explicitly without KKT differentiation:
3. Surrogate cost: $c_SPO = 2y^* - y$
4. Solve LP with surrogate costs: $z^*_SPO = \operatorname{argmin}_z (-c_SPO^T z)$ subject to battery constraints
5. Gradient: $dL_SPO+/dy^* = 2 * (z^*_SPO - z^*(y^*))$

Requires only two LP solves and a subtraction — no matrix inversions.

Metric	QP regularisation ($\gamma = 0.1$)	SPO+
Capture ratio (test)	0.74	0.75
Pinball loss (test)	\$19.8/MWh	\$20.1/MWh
Training wall time (10 epochs)	8 min	6 min
Hyperparameters to tune	γ , learning rate	Learning rate only

SPO+ advantage: +0.01 CR, 25% faster training, one fewer hyperparameter

The main advantage of SPO+ is practical: one fewer hyperparameter and the deployment LP is identical to the training LP. The two losses track each other loosely during training (SPO+ is an upper bound on regret). Speed advantage grows with problem size (longer horizons, FCAS co-optimisation).

Glossary

Term	Definition
Decision-focused learning	Train forecaster via downstream decision quality
SPO	Smart predict-then-optimize (Elmachtoub & Grigas)
SPO+ loss	Convex surrogate for decision regret
Decision regret	Revenue gap vs perfect-foresight dispatch
Diff. optimisation layer	Solves optimisation fwd; implicit diff bwd
Implicit differentiation	Gradients of optimal solution via KKT diff
Argmin differentiation	Gradient of argmin of parameterised program
Quadratic regularisation	Quadratic penalty → smooth, differentiable dispatch
KKT conditions	Optimality conditions for constrained convex programs
Piecewise-constant soln	LP dispatch jumps at thresholds; zero grad elsewhere
Pre-training	Init on standard loss before decision-focused loss
cvxpylayers	Differentiable CVXPY layers (PyTorch/JAX)
qpth	Differentiable QP solver for PyTorch
Price taker	Participant whose actions don't affect prices

Capture ratio	Actual revenue / perfect-foresight revenue
---------------	--

Summary

Decision-focused learning closes the gap between what the forecaster optimises (accuracy) and what the trading desk is paid for (capture ratio). The two-stage predict-then-optimize approach — used throughout Chapters 7–10 — decouples the forecaster from the dispatch problem, training the model to minimise pinball loss or MAE without knowledge of how the forecast will be used. Decision-focused learning reconnects the two stages by training the forecaster directly on decision regret: the revenue gap between the forecast-driven dispatch and the perfect-foresight dispatch. The technical challenge is backpropagating through the dispatch LP, which requires computing the gradient of an argmin. For linear programs, this gradient is zero almost everywhere because the optimal dispatch is a piecewise-constant function of the forecast. Two fixes restore gradient signal: quadratic regularisation converts the LP into a QP with smooth, differentiable solutions, while the SPO+ surrogate loss of Elmachtoub and Grigas provides a convex upper bound on regret with non-zero gradients everywhere. Both fixes are implemented in standard differentiable optimisation libraries (cvxpylayers, qpth). The training loop pre-trains the forecaster on pinball loss for a warm start, then fine-tunes on decision regret through the differentiable dispatch layer. For the Mannum BESS (100 MW / 200 MWh, SA1), the decision-focused model achieves a capture ratio of 0.74 versus 0.68 for the two-stage baseline — a 6 percentage-point improvement worth approximately \$700K per year in additional arbitrage revenue — despite having slightly worse MAE and pinball loss. The improvement concentrates on volatile days where price ranking determines revenue, and the model achieves it by reallocating forecast accuracy from periods where the battery is idle (overnight, midday) to periods where the dispatch decision is sensitive to the price (afternoon–evening peaks). Decision-focused learning does not replace the convex dispatch LP from Chapter 9 — it uses the LP as a differentiable subroutine during training and as the standard dispatch engine in deployment. It sits in the methods hierarchy between convex optimisation (the core) and reinforcement learning (used only for the price-maker emulator from Chapter 12), and extends naturally to FCAS-aware dispatch from Chapter 11.